

TP03 : Les algorithmes récurrents et arithmétiques

Adrien ESCOURROU

December 1, 2025

INTRODUCTION

Ce TP03 s'intéresse à deux familles d'algorithmes qui apparaissent partout en informatique et en mathématiques : les récurrences et les méthodes arithmétiques. Les suites et les relations de récurrence permettent de décrire des processus pas à pas et d'estimer leur comportement, tandis que les algorithmes arithmétiques couvrent les opérations numériques essentielles (addition, multiplication, divisions, familiarisation avec les algorithmes d'optimisation et de complexité).

Rappels essentiels

- Un algorithme récurrent d'ordre 1 récupère les éléments des précédents résultats, exemple :
 - Une valeur donnée de compteur i , la valeur de la somme est égale à la valeur ancienne incrementée de la valeur de i ($s \leftarrow s + i$).
 - Une suite mathématiques, où l'on récupère le résultat précédent $U_n = U_{n-1}$ (+ autres)
- Un algorithme récurrent d'ordre 2 récupère les éléments des 2 précédents résultats, exemple :
 - Une suite mathématiques, où l'on récupère les résultats précédent $U_n = U_{n-2} + U_{n-1}$ (+ autres)
- Pour convertir un nombre décimal en base X , on suit les étapes suivantes :
 1. Diviser n par X .
 2. Garder le reste et remplacer n par $(n \text{ div } X)$.
 3. Répéter les étapes 1 et 2 jusqu'à avoir $n = 0$.
- La méthode d'Euclide repose sur le principe suivant :
 - Si $b \neq 0$, alors $\text{PGCD}(a, b) = \text{PGCD}(b, r)$ ou r est le reste de la division euclidienne de a par b ($0 \leq r < b$).
 - Si non, alors $\text{PGCD}(a, 0) = a$.
- La méthode du PPCM repose sur le principe suivant :
 1. Déterminer le minimum (min) et le maximum (max) entre a et b .

2. Calculer le reste r de la division de $\max$ par $\min$ ($r = \max \bmod \min$).
3. Si $r = 0$, alors le $\max$ est le PPCM.
4. Si non, chercher le multiple successif de $\max$ ($\max = \max + a + b - \min$), refaire l'algorithme à partir de l'étape 2 jusqu'à avoir un multiple de $\min$ ($r = 0$)

*Tous vos résultats devront être publiés sur votre répertoire GIT partagé avec votre professeur en TP01. Ici vous nommerez un répertoire **TP02** et annoterait chaque code avec l'exercice associé. Par exemple, pour l'exercice 3 vous publierez **Exo3.py** dans **TP02**.*

Exercice 1

1. Écrire un algorithme d'une fonction **somme** qui permet de calculer la somme de n premiers entiers.
2. Améliorez ensuite l'algorithme en nommant la fonction **sommeChoix**, qui permet de calculer et d'afficher la somme de n entiers saisis par l'utilisateur.

NB : Les deux versions doivent apparaître sous GIT.

Exercice 2

Écrire un algorithme d'une procédure qui permet de rechercher la plus petite et la plus grande valeur dans un tableau contenant n entiers. Traduire cet algorithme en Python.

*NB : Vous pouvez vérifier vos résultats grâce aux fonctions python **min** et **max**, mais vous devez réécrire ces fonctions dans cet exercice.*

Exercice 3

Écrire en Python une fonction **termeSuite** qui permet de calculer et d'afficher les n premiers termes de la suite U définie par :

$$U_0 = 3, \quad U_n = 2U_{n-1} - 4, \quad n \geq 1$$

Exercice 4

Écrire en Python une fonction **termeSuite2** qui permet de calculer et d'afficher la valeur de U_n de la suite U définie par :

$$U_0 = 3, U_1 = 1, \quad U_n = U_{n-1} + U_{n-2}, \quad n \geq 2$$

Exercice 5

En se basant sur la méthode d'Euclide, écrire un algorithme d'une fonction intitulée **pgcd** qui permet de calculer le PGCD de deux entiers. Traduire cet algorithme en Python.

Exercice 6

En se basant sur la méthode de calcul du PPCM, écrire un algorithme d'une fonction intitulée **ppcm** qui permet de calculer le PPCM de deux entiers. Traduire cet algorithme en Python.

Exercice 7

Écrire un algorithme et déduire la traduction en Python d'une fonction intitulée **premier** qui permet de vérifier si un entier n ($n > 1$) est premier ou non.

NB : Un entier supérieur à 1 est dit premier s'il n'est divisible que par 1 et par lui même

Exercice 8

Écrire un algorithme et déduire la traduction en Python d'une fonction intitulée **factoriel** qui permet de calculer la factorielle d'un entier n .

NB : Le factoriel de n est noté $n !$ ($n ! = n \times (n - 2) \times (n - 1) \times \dots \times 4 \times 3 \times 2 \times 1$).

Exercice 9

Écrire un algorithme et déduire la traduction en Python d'une fonction intitulée **convBinaire** qui permet de convertir un nombre entier positif n en binaire.

*NB : Vous pouvez vérifier vos résultats grâce à la fonction python **bin**, mais vous devez réécrire cette fonction dans cet exercice.*

Exercice 10

Écrire un algorithme et déduire la traduction en Python d'une fonction intitulée **convHexa** qui permet de convertir un nombre entier positif n en hexadécimal.

*NB : Vous pouvez vérifier vos résultats grâce à la fonction python **hex**, mais vous devez réécrire cette fonction dans cet exercice.*

Conclusion

Bravo, vous avez vu la majorité des algorithmes standards en Python. Merci de tout publier sur GIT.